분야 AI/SW 기초 구분 데이터베이스와 백엔드 학습시간 60시간 필수/선택 선택

로그인이 되고 회원끼리 연결되는 웹 서비스 만들기

문제기술

기술적 설명



1. 미션 소개

로그인 구현했는데 권한 없는 사람도 들어가지던 경험, 한 번쯤 있으실 겁니다. 인증과 인가는 다릅니다. FastAPI의 의존성 주입(Depends)이 그 둘을 어떻게 분리하는지 이해하면, 보안 구멍이 어디서 생기는지 보이기 시작합니다.

인증, 인가 통합과 함께, 연관관계 기반의 비즈니스 로직까지 확장하는 것이 이번 목표입니다. 실제 웹 서비스에서 "로그인"은 단순한 기능이 아니라, 누가 어떤 기능에 접근할 수 있는지를 결정하는 핵심 구조입니다. FastAPI에서는 의존성 주입(Depends)과 미들웨어를 활용하여 인증·인가를 체계적으로 구현할 수 있습니다.

이 미션에서는 이전 미션에서 만든 웹 애플리케이션을 기반으로, 사용자 인증(로그인/로그아웃)과 접근 제어(인가)를 서비스에 통합합니다. 또한, 여러 모델이 관계를 맺는 도메인 구조를 설계하고, 단순 CRUD를 넘어 "상태가 변화하는 비즈니스 로직"을 중심으로 서비스를 확장합니다.

이 경험은 이후 AI/SW 응용 도메인 미션에서 FastAPI 기반 API 서버를 구축할 때 인증·인가 설계와 복잡한 도메인 모델링의 기반이 됩니다.

2. 최종 결과물

다음 기능이 정상 동작하는 웹 서비스 1개를 완성한다. (이 서비스는 이전 미션에서 구현한 웹 애플리케이션을 기반으로 확장한다.)

1. **FastAPI 기반 사용자 인증**

- 사용자는 로그인 / 로그아웃을 할 수 있다.
- 로그인 상태에 따라 화면(UI)과 접근 가능 기능이 달라진다.

2. 접근 제어(인가)

- 로그인하지 않은 사용자는 핵심 기능 화면에 접근할 수 없다.
- 비로그인 상태에서 접근 시 로그인 화면 또는 안내 화면으로 이동한다.

3. 연관관계가 있는 도메인 서비스

- 최소 3개 이상의 모델이 존재한다.
- 모델 간 연관관계(예: 1:N, N:1)가 존재하며, 화면에서 관계 데이터가 함께 출력된다.

4. 상태 변경 중심의 핵심 비즈니스 기능

- 단순 CRUD를 넘어 "상태가 바뀌는 기능"이 최소 1개 이상 포함된다.
- 상태 변경 전/후가 화면 또는 응답 결과로 명확히 구분된다.

5. 서비스로서의 통합 완성도

- 사용자가 다음 흐름을 끊김 없이 수행할 수 있어야 한다.
  - 로그인
  - 핵심 기능 진입
  - 데이터 생성 / 조회
  - 상태 변경
  - 결과 확인

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- FastAPI에서 인증/인가를 Depends 기반으로 어떤 흐름으로 처리하는지 설명할 수 있다.
- 로그인하지 않은 사용자의 요청이 왜 차단되는지, 어디에서 처리되는지 설명할 수 있다.
- SQLAlchemy 연관관계를 객체 관점에서 이해하고, 왜 그렇게 매핑했는지 설명할 수 있다.
- 단순 CRUD가 아니라 "상태 변경 로직"이 왜 중요하며, 이를 어디에 두었는지 설명할 수 있다.

- 이 서비스의 전체 구조(인증 → 도메인 → 화면 흐름)를 사용자 관점과 개발자 관점에서 설명할 수 있다.

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. 인증

- FastAPI의 의존성 주입(Depends)을 활용하여 인증 로직을 구현해야 한다.
- 인증 방식은 다음 중 하나를 선택한다.
    - **방식 A (세션 기반)**: 로그인 성공 시 서버 세션(쿠키)에 사용자 정보를 저장하고, 이후 요청에서 세션을 통해 인증 여부를 확인한다.
    - **방식 B (JWT 기반)**: 로그인 성공 시 JWT 를 발급하고, 이후 요청에서 토큰을 통해 인증 여부를 확인한다.
- 사용자는 로그인할 수 있어야 한다.
- 로그인 성공/실패 여부가 사용자에게 명확히 드러나야 한다.
- 사용자는 로그아웃할 수 있어야 한다.
- 테스트 계정은 인메모리(하드코딩) 또는 DB 저장 중 선택 가능하다. 단, 선택한 방식과 테스트 계정 정보(ID/PW)는 README에 명시해야 한다.
- 인증 구현을 위한 참고 구조: 인증이 필요 없는 경로와 필요한 경로를 구분하는 미들웨어 또는 Depends 함수를 별도 모듈(auth/)로 분리하여 구현한다.
    - 참고 키워드: OAuth2PasswordBearer , OAuth2PasswordRequestForm , python-jose , passlib , starlette.middleware.sessions

2. 접근 제어(인가)

- 로그인하지 않은 사용자는 핵심 기능 화면에 접근할 수 없어야 한다.
- 비로그인 사용자가 보호 자원에 접근할 경우, 로그인 페이지로 이동하거나 접근 차단 안내를 확인할 수 있어야 한다.
- 공개 페이지와 보호 페이지가 명확히 구분되어야 한다.
    - 예: / , /login , /signup 등은 공개, /app/** 또는 도메인 기능 경로는 보호(로그인 필수)
- 공개/보호 경로 구분 정책은 README에 표 형태로 정리하여 포함해야 한다.

3. 인증 상태에 따른 UI 변화

- 로그인 상태일 때와 아닐 때, 화면 요소가 달라져야 한다.
    - 예: 로그인 전에는 [로그인] 버튼, 로그인 후에는 "OOO님 환영합니다" + [로그아웃] 버튼

4. 도메인 모델 및 연관관계

- SQLAlchemy ORM 모델은 최소 3개 이상 존재해야 한다.
- 모델 간 1:N 또는 N:1 연관관계가 최소 2개 이상 존재해야 한다.
- 양방향 연관관계(`relationship` + `back_populates`)는 최소 1개 이상 포함되어야 한다.
- N:M 관계는 이 미션에서 다루지 않는다. (필요 시 중간 모델로 풀어서 구현)
- 연관관계를 활용한 데이터 출력이 화면에 포함되어야 한다.
    - 예: 사용자 상세 화면에서 사용자의 글 목록 출력
- 부모 모델 삭제 시 자식 모델 처리 정책(예: `cascade="all, delete-orphan"` 또는 수동 삭제)을 비즈니스 로직에 맞게 선택하고, 해당 이유를 코드 주석으로 명시해야 한다.
- "서비스 주제"는 학습자가 직접 정하되, 3개 이상의 모델과 상태 변경 로직이 포함될 수 있는 수준으로 선택한다. 아래는 참고할 수 있는 주제 예시이다.
    - 도서 대여 서비스: 회원, 도서, 대여기록 (대여중 → 반납완료)
    - 주문 관리 서비스: 회원, 상품, 주문 (주문접수 → 배송중 → 배송완료 / 주문취소)
    - 예약 시스템: 회원, 서비스, 예약 (예약대기 → 예약확정 → 예약취소)
    - 할 일 관리 서비스: 회원, 프로젝트, 할일 (진행중 → 완료)
    - 게시판 서비스: 회원, 게시판, 게시글 (공개 → 비공개 / 삭제)

5. 핵심 비즈니스 기능

- 상태가 변경되는 비즈니스 기능이 최소 1개 이상 존재해야 한다.
    - 예: 주문 → 취소, 할 일 → 완료/미완료
- 상태 변경 결과가 화면 또는 응답에서 확인 가능해야 한다.

6. 데이터 저장

- 모든 핵심 데이터는 SQLAlchemy ORM + 관계형 데이터베이스(SQLite 또는 PostgreSQL)에 저장되어야 한다.
- 연관관계 데이터 역시 DB 기준으로 조회되어야 한다.

7. 화면 흐름 통합

- 모든 주요 화면은 Jinja2 템플릿 기반 SSR로 렌더링되어야 한다.
- 사용자는 다음 흐름을 끊김 없이 수행할 수 있어야 한다.
    - 로그인
    - 핵심 기능 화면 진입
    - 데이터 생성/조회
    - 상태 변경 수행
    - 결과 확인

8. 코드 구조

- 인증, 도메인 로직, 데이터 접근, 화면 처리가 역할에 맞게 분리되어야 한다.
- 특정 기능이 한 파일 또는 한 계층에 과도하게 몰려 있지 않아야 한다.
- 최소한 다음 역할 분리를 갖추어야 한다.
    - auth/ (인증·인가 로직)
    - routers/ (요청/응답 처리)
    - services/ (비즈니스 로직)
    - repositories/ (데이터 접근)
    - models/ (SQLAlchemy ORM 모델)
    - templates/ (Jinja2 템플릿)

9. 제출물 구성

- 전체 프로젝트 소스코드 (GitHub 저장소 링크)
- README.md 파일에 다음 항목이 반드시 포함되어야 한다.
    - Python 버전 및 주요 패키지 버전
    - 가상환경 생성, 패키지 설치, 서버 실행 방법
    - 테스트 계정 정보 (ID/PW)
    - 공개/보호 경로 정책 표
    - 주요 기능 경로 목록
    - 인증 방식(세션/JWT) 선택 사유 요약
- (선택) 주요 화면 스크린샷 또는 시연 영상

5. 보너스 과제 (선택)

1. 전역 예외 처리

- 잘못된 접근, 없는 데이터 요청 등 실패 상황을 공통으로 처리하고, 사용자에게 일관된 방식으로 안내한다.
- 배움 포인트: FastAPI의 `@app.exception_handler` 와 커스텀 예외 클래스를 경험한다.

2. 검색 / 필터 기능

- 목록 데이터 또는 연관관계 데이터에 대해 조건 검색 또는 필터 기능을 1개 이상 추가한다.
- 배움 포인트: 쿼리 파라미터 기반 필터링과 SQLAlchemy의 동적 쿼리 구성을 경험한다.

3. OAuth2 소셜 로그인

- Google, GitHub 등 외부 소셜 계정을 이용해 로그인할 수 있도록 인증 흐름을 확장한다.
- 배움 포인트: OAuth2 Authorization Code Flow와 외부 IdP 연동을 경험한다.

4. 비밀번호 해싱 및 회원가입

- 사용자가 직접 회원가입할 수 있고, 비밀번호가 해싱되어 DB에 저장되도록 구현한다.
- 배움 포인트: `passlib` / `bcrypt` 를 활용한 비밀번호 보안 처리를 경험한다.

5. 외부 배포

- Render, Railway 등 클라우드 환경에 서비스를 배포하여 외부에서 접속 가능한 URL을 제출한다.
- 배움 포인트: Python 웹 서비스의 클라우드 배포 과정과 환경 변수 관리를 경험한다.

개발환경

6. 개발 환경

- Python 3.10 이상

제약조건

7. 제약 사항

- 의존성
  - 핵심 패키지: `fastapi`, `uvicorn`, `sqlalchemy`, `jinja2`, `python-multipart`
  - 인증 관련 패키지(선택한 방식에 따라): `python-jose`, `passlib`, `bcrypt`, `itsdangerous`, `starlette`
  - 위 범위를 벗어나는 외부 라이브러리는 추가하지 않는다.
- 구조
  - 인증, 도메인, 데이터 접근, 화면 처리는 역할에 맞게 분리한다.
  - 모델 간 연관관계는 코드에서 명확히 드러나야 한다.
- 기능 범위
  - 복잡한 권한 체계(다단계 역할 기반 접근 제어)는 요구하지 않는다. 로그인/비로그인 구분만으로 충분하다.
  - OAuth2 소셜 로그인은 보너스 과제에서 선택적으로 다룬다.

Test Case

8. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 문구, 디자인은 얼마든지 달라도 된다.

- **로그인 전 화면 예시**

```
CSS
서비스 이름

[로그인] 버튼이 보인다.
핵심 기능 메뉴는 보이지 않거나 클릭 시 로그인 페이지로 이동한다.
```

- **로그인 후 화면 예시**

```
서비스 이름

"OOO님 환영합니다" 문구가 보인다.
[로그아웃] 버튼이 보인다.
핵심 기능 메뉴가 활성화되어 있다.
```

- **연관관계 데이터 출력 예시**
  - 사용자 상세 화면에서 사용자의 등록 데이터 목록이 함께 보인다.
  - 주문 상세 화면에서 주문 항목들이 함께 보인다.
- **상태 변경 예시**
  - "예약 완료 → 예약 취소"
  - "주문 생성 → 주문 취소"
  - 상태 변경 결과가 화면에서 확인 가능하다.